

NoSQL

Week 6 – Part 1 – Introduction to NoSQL

CS 457 - L1 Data Science

Zeesham Rasheed

What is NoSQL



- Stands for No SQL or **Not Only SQL**
- Class of **non-relational** data storage systems
- Usually **do not require a fixed table schema**,
- Used for Text, Web, emails, documents, tweets..."**unstructured data**"

RDBMS/SQL vs NoSQL



NoSQL:

Includes **non-relational**

Documents, key-value pairs, graphs, wide columns...**and tables**

“**Dynamic**” schema

Horizontal scaling (usually)

Some SQL-like, plus other languages

NO STANDARD LANGUAGE

MongoDB, Cassandra, Hbase, CouchDB, ...

Typical Uses:

Data Analytics

Non-transactional

RDBMS/SQL:

Relational model

Table based

Predefined schema

Vertical scaling (usually)

SQL language family

STANDARDIZED

Oracle, MySQL, DB2, SQLserver, ...

Typical Uses:

Large-scale **transactional**

Finance, for example

Banks, credit cards, stock trading

Common Advantages of NoSQL Systems



- Cheap, easy to implement (**open source**)
- Data **replicated** to many multiple nodes, which:
 - Are identical and are "fault-tolerant"
 - Can be partitioned – after a data write:
 - Latest version is on at least one node
 - Can be replicated to other nodes
 - **No single point of failure**
- Easy to distribute
- No schema required (generally)

Some NoSQL Databases



MongoDB

<https://www.mongodb.com/>

Couchbase

<https://www.couchbase.com/>

OrientDB

<http://orientdb.com/orientdb/>

Cassandra

<http://cassandra.apache.org/>

Hbase

<https://projects.apache.org/project.html?hbase>

Oracle NoSQL

<https://www.oracle.com/database/nosql/index.html>

NoSQL Distinguishing Characteristics



Large data volumes

- Like Google's "bigdata"

Scalable replication and distribution

- Potentially **thousands of machines**
- Potentially **distributed around the world**

Queries need to return answers **quickly**

Mostly query, few updates

Asynchronous Inserts & Updates

Schema-less (some say schema-free)

ACID transaction properties are generally **not used**

- BASE: **BASE** (read-mostly) Basic Availability, Soft state, Eventual
- Especially Eventually Consistency

Open Source development

Relational Databases are Awesome



- Relational Databases solve most data problems
 - **Persistence**
We can store data, and it will remain stored!
 - **Integration**
We can integrate lots of different apps through a central DB
 - **SQL**
Standard(ish), well understand, very expressive
 - **Transactions**
ACID transactions, strong consistency

- A few key trends and issues...
 - ...In **use cases**
 - ...In **technology**
- Key trends include:
 - Increasing volume of **data and traffic**
 - More complex **data connectedness**
- Key Issues include:
 - The **impedance mismatch** problem
 - is a set of conceptual and technical difficulties that are often encountered when a relational database management system (RDBMS) is being used by a program, due to structure complexity.

Issue – Impedance Mismatch



Player Table

ID: 1001

USER: Steve

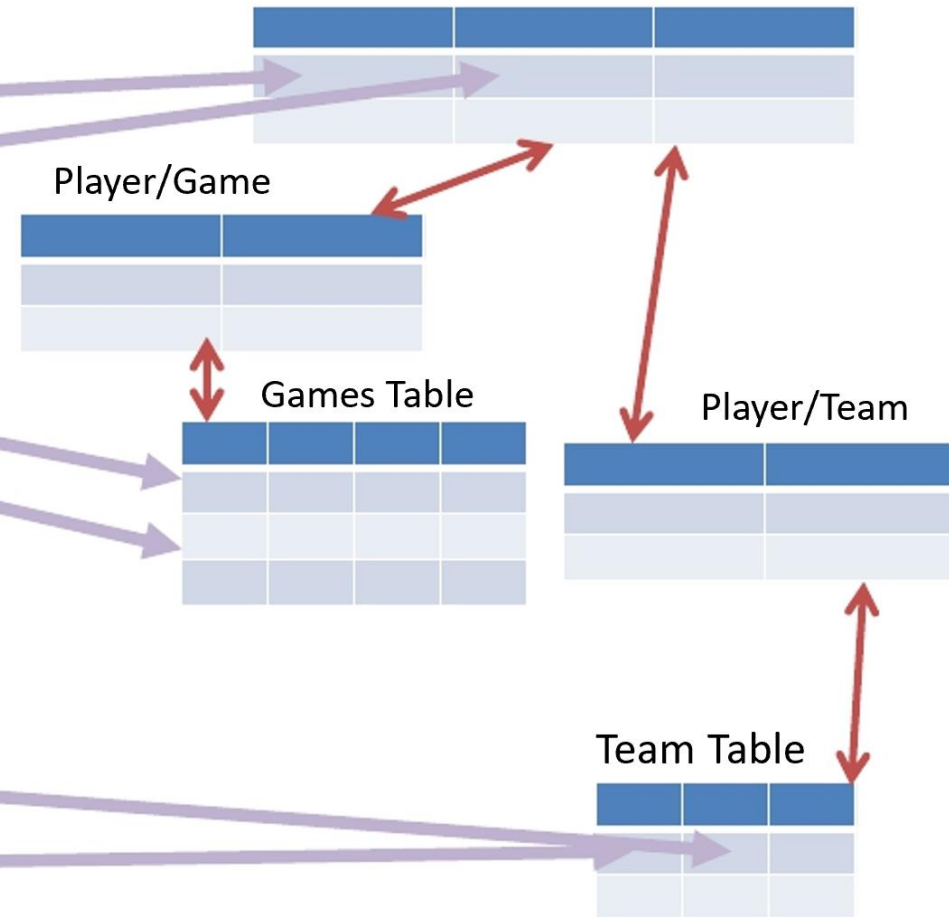
Games Played:

Date	Res	K	D	A
01/04/2009	WIN	20	2	10
01/05/2009	LOOSE	5	22	3

Teams:

Name: Killer Bee Keepers

Icon:
<http://imgur.com/a/...>



- We (the world) are
... Creating, Storing, Processing...
... more data than ever before!

“From **2005 to 2020**, the digital universe will grow by a factor of 300, from **130 exabytes to 40,000 exabytes**, or **40 trillion gigabytes** (more than 5,200 gigabytes for every man, woman, and child in 2020).

From now until 2020, the digital universe will about **double every two years.**”

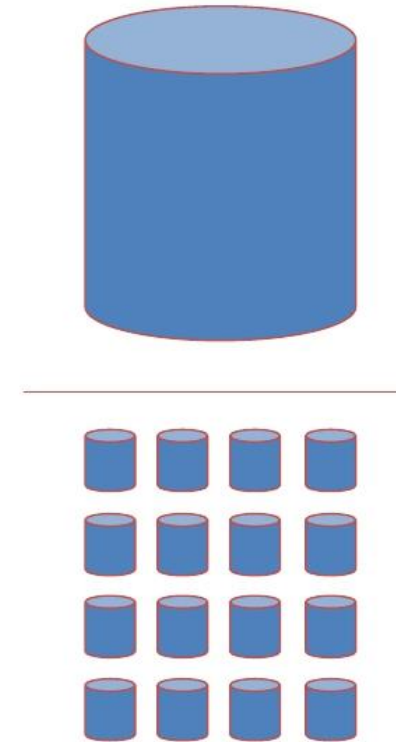
- **IDC** – *The Digital Universe in 2020*

<http://www.emc.com/leadership/digital-universe/index.htm>

Dealing with data size Trends



- Two options when dealing with these trends:
- Build **Bigger** Database machines
 - This can be **expensive**
 - **Fundamental limits** to machine size
- Build **Cluster** of smaller machines
 - Lots of **small machines** (*commodity machines*)
 - Each machine is cheap, potentially unreliable
 - Needs a DBMS which **understands cluster**



Relational Databases Limitations



- RDBMS have **fundamental issues**
 - Designed to work on **single, large machines**
 - Difficult to **distribute** effectively
 - Unnecessary **joins**

NoSQL - Not Only SQL



- NoSQL came to address
 - “**web-scale** problems”
 - **impedance mismatch** solution on the way
- Key attributes include:
 - **Non-Relational** (*Though they can be, but aren't good at it*)
 - **Schema Free** (*Except the implicit schema, application side*)
 - Inherently **Distributed** (*In different ways, Some more so than others*)
 - **Open Source** (*e.g. Oracle's NoSQL, MongoDB*)

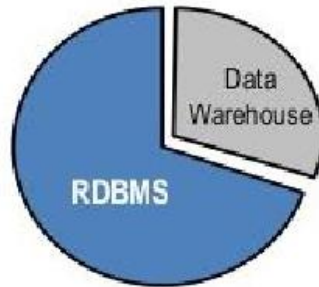
Three Eras of Databases



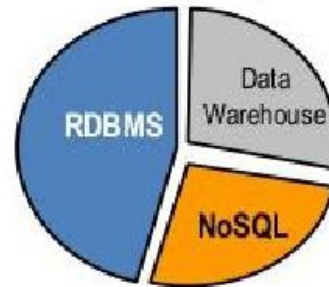
- RDBMS for transactions
- Data Warehouse for analytics
- NoSQL for scalability



1985-1995



1995-2010



2010-Now

Transactions – ACID Properties



- **Atomic** – All of the work in a transaction completes (commit) or none of it completes
- **Consistent** – A transaction transforms the database from one consistent state to another consistent state. Consistency is defined in terms of constraints.
- **Isolated** – The results of any changes made during a transaction are not visible until the transaction has committed.
- **Durable** – The results of a committed transaction survive failures

Dropping Relaxing ACID



- ACID is a big deal in traditional RDBMS
 - Broken transactions in Banking is a **big deal**
 - Money leaving your account but not entering the seller's account is a **big deal**
- BUT in some situations, the use case doesn't need all or some of ACID
 - Seeing an old version of a **facebook post**
 - An **amazon shopping cart** forgetting your items
 - A **tweet or two out** of the 138 million per day being lost

- When no updates occur for a long period of time
- Known as BASE (**B**asically **A**vailable, **S**oft state, **E**ventual consistency), as opposed to ACID
 - Soft state: copies of a data item may be inconsistent
 - Eventually Consistent – copies becomes consistent at some later time if there are no more updates to that data item

Acronym contrived to be the opposite of ACID

- **B**asically **A**vailable,
- **S**oft state,
- **E**ventually Consistent

Characteristics

- Weak consistency – inconsistent data is accepted
- Availability first
- Best effort
- Approximate answers OK
- Simpler and faster

Common Advantages of NoSQL Systems



- Cheap, easy to implement (open source)
- Data are replicated to multiple nodes (therefore identical and fault-tolerant) and can be partitioned
- When data is written, the latest version is on at least one node and then replicated to other nodes
- No single point of failure
 - Easy to distribute
 - Don't require a schema

What does NoSQL Not Provide?



- Joins (not as straight forward as in SQL)
- Group By (not as straight forward as in SQL)
- ACID transactions
- Integration with applications that are based on SQL

Should I be using NoSQL Databases?



- NoSQL Data storage systems makes sense for applications that need to deal with very large semi-structured data
 - ❑ Log Analysis
 - ❑ Social Networking Feeds

End of Part 1



NoSQL

Week 6 – Part 2 – Document Databases

CS 457 - L1 Data Science

Zeesham Rasheed

Document Database - Basics



```
{
  "_id": "1",
  "name": "steve",
  "games_owned": [
    {"name": "Super Meat Boy"},
    {"name": "FTL"},
  ],
}
```



```
{
  "_id": "2",
  "name": "darren",
  "handle": "zerocool",
  "games_owned": [
    {"name": "FTL"},
    {"name": "Assassin's Creed 3", "dev": "ubisoft"},
  ],
}
```


MongoDB – Document Based Database



- Data are organized in collections. A collection stores a set of documents.
- Collection are like table and documents are like record
 - but: each document can have a different set of attributes even in the same collection
 - Semi-structured schema!
- Only requirement: every document should have an “_id” field (default object id by mongoDB)



MongoDB - Data Example



```
"_id": ObjectId("4efa8d2b7d284dad101e4bc9"),  
"Last Name": " Cousteau",  
"First Name": " Jacques-Yves",  
"Date of Birth": "06-1-1910" },
```

```
"_id": ObjectId("4efa8d2b7d284dad101e4bc7"),  
"Last Name": "PELLERIN",  
"First Name": "Franck",  
"Date of Birth": "09-19-1983",  
"Address": "1 chemin des Loges",  
"City": "VERSAILLES" }
```

- **Problem:**

- Product catalogs store different types of objects with different sets of attributes.
- This is not easily done within the relational model, need a more “flexible schema”

- **Relational Solutions**

- Create a table for each product category
- Put everything in one table
- Use inheritance
- Entity-Attribute-Value
- Put everything in a BLOB

RDBMS (1): Table per Product



```
CREATE TABLE `product_audio_album`  
  ( `sku` char(8) NOT NULL, ...  
    `artist` varchar(255) DEFAULT NULL,  
    `genre_0` varchar(255) DEFAULT NULL,  
    `genre_1` varchar(255) DEFAULT NULL, ...  
    PRIMARY KEY(`sku`)) ...
```

```
CREATE TABLE `product_film`  
  ( `sku` char(8) NOT NULL, ...  
    `title` varchar(255) DEFAULT NULL,  
    `rating` char(8) DEFAULT NULL, ...  
    PRIMARY KEY(`sku`)) ...
```

RDBMS (2): Single table for all



```
CREATE TABLE `product`  
  ( `sku` char(8) NOT NULL, ...  
    `artist` varchar(255) DEFAULT NULL,  
    `genre_0` varchar(255) DEFAULT NULL,  
    `genre_1` varchar(255) DEFAULT NULL, ...  
    `title` varchar(255) DEFAULT NULL,  
    `rating` char(8) DEFAULT NULL, ...  
    PRIMARY KEY(`sku`))
```

RDBMS (3): Relation



```
CREATE TABLE `product`  
  ( `sku` char(8) NOT NULL,  
    `title` varchar(255) DEFAULT NULL,  
    `description` varchar(255) DEFAULT NULL,  
    `price`, ...  
    PRIMARY KEY(`sku`))
```

```
CREATE TABLE `product_audio_album`  
  ( `sku` char(8) NOT NULL, ...  
    `artist` varchar(255) DEFAULT NULL,  
    `genre_0` varchar(255) DEFAULT NULL,  
    `genre_1` varchar(255) DEFAULT NULL, ...  
    PRIMARY KEY(`sku`),  
    FOREIGN KEY(`sku`) REFERENCES `product`(`sku`))
```

- A “collection” can contain heterogeneous “documents”, e.g. for an audio album we could store

```
{ sku: "00e8da9b",  
  type: "Audio Album",  
  title: "A Love Supreme",  
  description: "by John Coltrane",  
  shipping: { weight: 6,  
             dimensions: { width: 10, height: 10, depth: 1 } },  
  pricing: { list: 1200, retail: 1100, savings: 100},  
  details: { title: "A Love Supreme [Original Recording]",  
            artist: "John Coltrane",  
            genre: [ "Jazz", "General" ] }  
}
```

MongoDB Document Example



```
db.mycol.insert({  
    first: 'john',  
    last: 'smith',  
    gender: 'm',  
    hair_color: 'brown',  
    occupation: 'developer',  
    nationality: 'american'  
});
```

- “first”, “last”, “gender”, etc. are arbitrary tags.
- Database is schema-less

- MongoDB (from "**humongous**") is a scalable, high-performance, open source, document-oriented database.
 - Fast querying & In-place updates
 - Full Secondary Index Support
 - Replication & High Availability
 - Auto-Sharding
- Currently used in a number of different applications
 - Craigslist, ebay, New York Times, Shutterfly, Chicago Tribune, Github, Disney

- A MongoDB deployment hosts a number of databases. A **database** holds a set of collections. A **collection** holds a set of documents. A **document** is a set of key-value pairs.

RDBMS	MongoDB
Table	Collection
Row(s)	JSON Document
Index	Index
Join	Embedding & Linking
Partition	Shard
Partition Key	Shard Key

End of Part 2



NoSQL

Week 6 – Part 3 – MongoDB Example
CS 457 - L1 Data Science

Zeesham Rasheed

Basic Data Types



- Null
- Boolean
- Integer (32- and 64-bit)
- Floating point
- String
- ObjectId
- Date
- Code (JavaScript)
- Array

MongoDB

- Uses command line syntax
- “Databases” hold “collections” (somewhat like tables)
- **Create a database:**
 - use {dbname}
 - use students
- **Insert data**
 - > db.{collection}.insert({dataname: value, ... , dataname: value})
 - > db.students.insert({name: "John Smith", course: "AIT580"})
 - > db.students.insert({name: "Jane Jones", type: "Graduate"})

Collection will be created if doesn't exist

More Example Queries



```
db.students.insert({name: "John Smith", course: "AIT580"})
db.students.insert({name: "Jane Jones", type: "Graduate"})
db.students.insert({name: "Mary Miller", type: "Undergraduate"})
db.students.insert({name: "Ben Adams", course: "AIT580", type: "Graduate"})
db.students.count()
db.students.count({type: "Graduate"})
db.students.find()
db.students.find({type: "Graduate"})
db.students.find().sort({name: 1})
```

Automatically creates index (_id) for each record

Core MongoDB Operations



- CRUD: *create, read, update, and delete*
- Insert
 - One at a time: `db.people.insert(mydoc)`
- Delete
 - Documents that match some predicate, e.g. to remove the document just added:
`db.mydb.remove({"_id": 1})`
 - All documents in a collection: `db.mydb.remove()`
 - The collection still remains, with indexes
 - Remove a collection (faster): `db.people.drop()`

Sample MongoDB Document



```
mydoc = {  
  _id: 1,  
  name: { first: "John", last: "Backus" },  
  birthyear: integer,  
  contribs: [ "Fortran", "ALGOL", "Backus-Naur Form", "FP" ],  
  awards: [ { award_id: "NMS001",  
              year: 1975 },  
            { award_id: "TA99",  
              year: 1977 } ]  
}
```

Always indexed. Can be any data type other than an array

Array of documents

- Use find/findOne functions and a query document
- Ranges, set inclusion, inequalities using \$ conditionals
- Complex queries using \$where and \$elemMatch clause

find()



```
db.posts.find()  
    returns entire collection in posts
```

```
db.posts.find({"author": "Kevin", "birthyear": 1980})
```

OUTPUT:

```
{  
  _id : ObjectId("4c4ba5c0672c685e5e8aabf3"),  
  author : "Kevin",  
  date : Date("February 2, 2012"),  
  birthyear: 1980,  
  text : "About MongoDB...",  
  tags : [ "tech", "databases" ]  
}
```

Ranges, Negation, OR-clauses



- **Comparison operators: \$lt, \$lte, \$gt, \$gte**
 - `db.posts.find({"birthyear": {"$gte": 1970, "$lte": 1990}})`
- **Negation: \$ne**
 - `db.posts.find({"birthyear": {"$ne": 1982}})`
- **OR queries: \$in (single key), \$or (different keys)**
 - `db.posts.find({"birthyear": {"$in": [1982, 1985]}})`
 - `db.posts.find({"$or": [{"birthyear": 1982}, {"name": "John"}]})`

- `db.posts.find({"tags": "tech"})`
 - Print complete information about posts which are tagged "tech"
- `db.posts.find({"tags": {$all: ["tech", "databases"]}, {"author":1, "tags":1})`
 - Print author and tags of posts which are tagged with both "tech" and "databases" (among other things)
 - Contrast this with:
`db.posts.find({"tags": ["databases", "tech"]})`

Limits, Skips, Sort, Count



- `db.posts.find().limit(3)`
 - Limits the number of results to 3
- `db.posts.find().skip(3)`
 - Skips the first three results and returns the rest
- `db.posts.find().sort({"author":1, "title": -1})`
 - Sorts by author ascending (1) and title descending (-1)
- `db.people.find(...).count()`
 - Counts the number of documents in the people collection matching the `find(...)`

Also Inserting Records As Variables



award1=

```
{_id: "NMS001",  
  title: "National Medal of Science" ,  
  by: "National Science Foundation"}
```

award2=

```
{_id: "TA99",  
  title: "Turing Award",  
  by: "ACM" }
```

```
db.awards.insert(award1)
```

```
db.awards.insert(award2)
```

Summary of MongoDB



- MongoDB is an example of a document-oriented NoSQL solution
- The query language is limited, and oriented around “collection” (relation)
 - Joins are done via a query language
- The power of the solution lies in the distributed, parallel nature of query processing
 - Replication and sharding

- NoSQL databases reject:
 - Overhead of ACID transactions
 - “Complexity” of SQL
 - Burden of up-front schema design
 - Declarative query expression

SQL vs. NoSQL Summary



- NoSQL Database
 - No predefined Schema
 - Per-product definition and interface language
 - Getting an answer quickly is more important than getting a correct answer
 - Query standards have yet to be established
- SQL Databases
 - Predefined Schema
 - Standard definition and interface language
 - Tight consistency
 - Fine-grained security on columns/rows using views
 - Well defined semantics

- NoSQL means “Not Only SQL”
- Relational databases will continue to be an appropriate solution to many business problems for the foreseeable future.
- RDBMSs are continuing to evolve and are making it possible to relax ACID requirements and manage document-oriented structures.
- But there are situations where relational databases aren't the best match for a business problem.
- Need to consider nature of data, business requirements, and tradeoffs between consistency, availability and scalability

- NoSQL means “Not Only SQL”
- Relational databases will continue to be an appropriate solution to many business problems for the foreseeable future.
- RDBMSs are continuing to evolve and are making it possible to relax ACID requirements and manage document-oriented structures.
- But there are situations where relational databases aren't the best match for a business problem.
 - Need to consider nature of data, business requirements, and trade-offs between consistency, availability and scalability

- RDBMS vs. NoSQL: How Do You Pick?
 - <http://www.zdnet.com/rdbms-vs-nosql-how-do-you-pick-7000020803/>
- SQL vs. NoSQL: Which is Better?
 - <http://news.dice.com/2012/07/16/sql-vs-nosql-which-is-better/>
 - Good summary on arguments from both sides

Web References



- “NoSQL -- Your Ultimate Guide to the Non - Relational Universe!”
<http://nosql-database.org/links.html>
- “NoSQL (RDBMS)”
<http://en.wikipedia.org/wiki/NoSQL>
- PODC Keynote, July 19, 2000. *Towards Robust. Distributed Systems*. Dr. Eric A. Brewer. Professor, UC Berkeley. Co-Founder & Chief Scientist, Inktomi .
www.eecs.berkeley.edu/~brewer/cs262b-2004/PODC-keynote.pdf
- “Brewer's CAP Theorem” posted by Julian Browne, January 11, 2009.
<http://www.julianbrowne.com/article/viewer/brewers-cap-theorem>
- “How to write a CV” Geek & Poke Cartoon
<http://geekandpoke.typepad.com/geekandpoke/2011/01/nosql.html>

Web References (2)



- “Exploring CouchDB: A document-oriented database for Web applications”, Joe Lennon, Software developer, Core International.
<http://www.ibm.com/developerworks/opensource/library/os-couchdb/index.html>
- “Graph Databases, NOSQL and Neo4j” Posted by Peter Neubauer on May 12, 2010 at: <http://www.infoq.com/articles/graph-nosql-neo4j>
- “Cassandra vs MongoDB vs CouchDB vs Redis vs Riak vs HBase comparison”, Kristóf Kovács. <http://kkovacs.eu/cassandra-vs-mongodb-vs-couchdb-vs-redis>
- “Distinguishing Two Major Types of Column-Stores” Posted by Daniel Abadi on March 29, 2010
http://dbmsmusings.blogspot.com/2010/03/distinguishing-two-major-types-of_29.html

Web References (3)



- “MapReduce: Simplified Data Processing on Large Clusters”, Jeffrey Dean and Sanjay Ghemawat, December 2004.
<http://labs.google.com/papers/mapreduce.html>
- “Scalable SQL”, ACM Queue, Michael Rys, April 19, 2011
<http://queue.acm.org/detail.cfm?id=1971597>
- “a practical guide to noSQL”, Posted by Denise Miura on March 17, 2011 at <http://blogs.marklogic.com/2011/03/17/a-practical-guide-to-nosql/>

- **Read the following manuals:**
- <http://docs.mongodb.org/manual/tutorial/getting-started/>
-
- <http://docs.mongodb.org/manual/reference/sql-comparison/>
-
- <http://docs.mongodb.org/v2.4/MongoDB-crud-guide.pdf>

- “CouchDB *The Definitive Guide*”, J. Chris Anderson, Jan Lehnardt and Noah Slater. O’Reilly Media Inc., Sebastopool, CA, USA.
2010
- “Hadoop *The Definitive Guide*”, Tom White. O’Reilly Media Inc., Sebastopool, CA, USA. 2011
- “MongoDB *The Definitive Guide*”, Kristina Chodorow and Michael Dirolf. O’Reilly Media Inc., Sebastopool, CA, USA.
2010
- “NoSQL Distilled,” P. Sadalage and M. Fowler, Addison-Wesley,
2012

Using NoSQL in MongoDB



CRUD (**C**reate, **R**ead, **U**ppdate, **D**eleate) Operators

<https://docs.mongodb.com/manual/crud/>

Aggregation Operators

- <https://docs.mongodb.com/manual/>

- Logical Operators

<https://docs.mongodb.com/manual/reference/operator/query-logical/>

Comparison Operators

<https://docs.mongodb.com/manual/reference/operator/query/>

MongoDB "Cheat Sheet":

<https://developers.redhat.com/promotions/mongodb-cheatsheet/>

- **Mongo Resources**
 - IDMA Chapter 7
 - https://info.jblearning.com/jams_data_links
 - <https://docs.mongodb.com/manual/tutorial/>

End of Part 3

